ADR 001: CRDT collaboration through a server relay

Status

Proposed on 14 October 2026; awaiting group review. Task 1 implements presence and conflict detection only. This records the proposed direction for Task 2, not an implemented CRDT engine.

Context

The final platform must preserve edits from multiple users despite concurrency, delay and reconnection. We need a design the group can explain and test. Document content, ephemeral presence and durable storage have different lifecycles.

Operational Transformation (OT) transforms operations relative to concurrent operations. A central ordering/versioning service can fit an online editor well, but transformation correctness and history/reconnect handling must remain consistent across the chosen text operations. ShareDB supplies an OT-based backend, so the alternative is a mature library, not a request to invent transforms from scratch.

A CRDT encodes mergeable updates with identifiers and causal information. Yjs offers shared types, editor bindings and transport providers. Under its algorithm's assumptions, replicas that eventually receive the same updates converge; the application must still deliver updates, authorize users, persist state and bound resource use. CRDT metadata and compaction complicate storage and debugging. It does not eliminate all user-visible edit surprises.

Decision

Evaluation criteria, in priority order: correctness under reconnection, availability of maintained text/editor bindings, testability, local reproducibility, and operational complexity. These are qualitative criteria, not measured benchmark scores. Both Yjs and ShareDB are credible: ShareDB supports offline change synchronization too, so offline support alone does not decide the choice. We favour Yjs because mergeable text updates and editor bindings fit the proposed future direction, while retaining a server for authorization and durability. If a group prototype shows materially better understanding or integration with ShareDB, revisit this proposal before Task 2.

Propose Yjs text CRDT for Task 2, connected through an authenticated client-server WebSocket relay. The relay is authoritative for access, admission and persistence, while CRDT merging determines text convergence. Keep presence ephemeral and separate from persistent document state.

Reject a peer-to-peer WebRTC mesh with cloud fallback for the initial product. It adds signalling, NAT traversal, relay fallback, peer lifecycle, access revocation and distributed persistence decisions before they serve an accepted requirement. A server relay is simpler to operate and observe locally, at the cost of a central availability dependency.

For Task 1, use plain text snapshots with a database version number. The save performs one conditional UPDATE and returns 409 for a stale version. This prevents silent overwrites but does not merge edits. The WebSocket service exchanges join/roster events and ping/pong only.

Consequences

• Future CRDT tests must simulate duplicate, reordered, delayed and dropped updates followed by reconnection, and assert convergence and durability.

• Persist CRDT updates or consistent snapshots and state vectors when Task 2 arrives; do not independently overwrite a plain-text snapshot while CRDT sessions are active.

• Define document authorization at both HTTP and socket boundaries. A convergent data structure does not provide access control.

• A single relay is sufficient for this milestone. Horizontal scaling needs coordinated rooms and shared durable state and is not implemented here.

• Current optimistic versioning protects saves but requires manual conflict reconciliation. The UI explains this limitation.

AI prompt appendix

Audit input on 21 October 2026: the user asked to "analyze all, we need strong project and fully fill the requirements and more for the task 1 project need to develop also need the text for group mates so they will see the project and understand so they could write documentation, share ideas release it on github and write theoretical part". In response, the assistant checked the specification, separated transport into presence.ts and expanded room/expiry/reconnect tests. This is an actual assistant interaction, not a substitute for the group's own targeted CRDT/OT research log.

Actual user prompts in this session included: "it is group project for Advances Software engineering" and "create repository for the beginning and do the task 1 and intro moments for the projcet", together with the supplied Task 1 specification. This ADR was drafted by an AI coding assistant from those inputs and checked against the primary sources below. No separate team research conversation or team approval has occurred in the available record.

Suggested follow-up prompt for the team (not yet run): "Compare Yjs and ShareDB for our Markdown editor under reconnect, dropped messages, access revocation and a single-server local deployment. Identify what each library does not solve and propose tests."

Critical commentary: Separating ephemeral presence from durable text and distinguishing convergence from access control are useful architectural insights. The assistant's recommendation remains a proposal without workload measurements, a Yjs prototype or evidence of the group's familiarity with either library. Treating 'CRDT' as a guarantee against all lost work would be superficial: update delivery, durable writes, migration and user intent still matter. The group should challenge this draft with a small reconnect experiment and record its own findings rather than present suggested prompts as completed research.

Primary references

• Yjs documentation: https://docs.yjs.dev/

• Yjs awareness and presence: https://docs.yjs.dev/getting-started/adding-awareness

• ShareDB introduction: https://share.github.io/sharedb/

ADR 002: TypeScript service, browser modules and SQLite

Status

Proposed on 14 October 2026; implemented in the initial skeleton and awaiting group acceptance.

Context

The marker must run the application on a commodity machine. Task 1 needs authentication, persistent document storage, Markdown rendering and genuine WebSockets. Later tasks change the parser and collaboration behavior, so transport, persistence and rendering need visible boundaries. A small team also needs a reproducible build and understandable tests.

Decision

Priorities are reproducible marker setup, understandable module boundaries, verifiable behavior and room for later changes. A framework is not currently used: this is an explicit no-framework frontend selection, not an omitted React implementation. The brief's examples should be clarified with the convenor if a framework is expected. A group decision to use React/Svelte/Vue should be recorded here and implemented through peer review rather than silently relabelling the existing UI.

Use Node.js 24 and TypeScript for the backend, Express for HTTP routing, the ws library for WebSockets, and SQLite through Node's built-in node:sqlite module. Use parameterized SQL, a versioned conditional update for save conflicts, WAL journaling and foreign keys. Store users, salted password hashes, session-token hashes and expiry, and titled documents with author references.

Use browser-native JavaScript modules and semantic HTML/CSS for the first editor. React, Svelte or Vue could improve state management in a larger application, but this milestone has a small number of screens and no requirement for a framework. Avoiding a frontend bundler keeps the walking skeleton inspectable. Revisit the frontend choice when introducing a CRDT editor binding; it is not an irreversible product-wide ban on frameworks.

Use marked for parsing and sanitize-html for output filtering, behind src/markdown.ts. Raw HTML must not become executable. Serve frontend and API from one origin to simplify session cookies and origin checks. Native WebSockets meet the required full-duplex channel without the additional application protocol supplied by Socket.IO; ws supplies server framing and control-frame handling.

Use pnpm with a committed lockfile. CI enforces ESLint, Prettier, strict backend TypeScript, Node test runner integration tests and compilation. The application works without cloud services. CodeRabbit is the proposed PR review integration, activated separately by the repository owner.

Consequences

• The 21 October audit moved WebSocket rooms, heartbeat and revocation into src/presence.ts, leaving HTTP handling in src/app.ts. This creates a concrete transport boundary; it does not imply distributed service isolation.

• SQLite gives a one-file local setup and transactions; concurrent writers serialize. It is appropriate for this small single-instance skeleton, not evidence that enterprise-scale throughput is solved.

• PostgreSQL would improve multi-service operations and scaling options but adds a service and setup burden. MongoDB's document model is unnecessary for this relational account/session/author model. Revisit storage when requirements justify it.

• A Node/TypeScript backend shares concepts with browser code. Go or Python are reasonable alternatives but add a language/toolchain for this team to maintain; this choice is provisional until the group confirms skills.

• Synchronous SQLite calls briefly block Node's event loop. Keep queries bounded and migrate or isolate work if measured load demands it. Password derivation uses asynchronous scrypt.

• The UI is linted JavaScript; backend TypeScript is strictly checked. Large frontend state changes may justify TypeScript and a framework later.

• Every signed-in user belongs to one shared workspace. Signup is open for local demonstration; production requires a separately designed admission/authorization policy.

• node:sqlite is version-sensitive, so Node 24 is explicitly required. Back up the data directory with the application stopped; do not copy an active database without its WAL state.

AI prompt appendix

Actual follow-up input on 21 October 2026 requested a full requirement analysis, stronger development, group handover and help with the theoretical part. The assistant found that its original startup instructions assumed the assistant's tool PATH matched the user's terminal. That assumption was wrong. The revised README explains independent Node/pnpm installation and the local runtime files are excluded from Git; this is a concrete example of why AI advice needs clean-machine validation.

Actual input: the user asked to "create repository for the beginning and do the task 1 and intro moments for the projcet" and supplied the Task 1 requirements. The assistant selected and implemented this stack from that context. No separate team prompt log, performance benchmark or completed team review was supplied.

Suggested follow-up prompt (not yet run): "Critique Node 24, SQLite, ws and a vanilla JavaScript frontend for our Task 1 skeleton. Compare PostgreSQL and React, identify upgrade boundaries, and challenge security and local reproducibility assumptions."

Critical commentary: The assistant's small local stack makes the requirement-to-code mapping easy to inspect and avoids unnecessary service setup. Separating persistence and Markdown rendering is useful for later changes. Its simplicity argument is incomplete without knowing team experience, expected workload and the next editor integration. A superficial claim that SQLite or scrypt alone makes the product production-ready would ignore authorization, signup admission, rate limiting across instances, recovery and operational monitoring. The group should validate the trade-off with a clean-clone rehearsal, inspect real CI evidence, and amend this record with its own reasoning.

Primary references

• Node SQLite API: https://nodejs.org/api/sqlite.html

• ws implementation and examples: https://github.com/websockets/ws

• Marked security guidance: https://marked.js.org/#usage

• CodeRabbit configuration: https://docs.coderabbit.ai/reference/configuration